

# The Speed Limit of Sorting

## And the Algorithm That Hits It

---

### Lecture 23

Data Structures and Algorithms

## Where We Left Off

---

Last lecture we compared all three simple sorts. They all share one painful truth:

| Algorithm      | Worst Case | Space  | Stable? |
|----------------|------------|--------|---------|
| Bubble Sort    | $O(n^2)$   | $O(1)$ | Yes     |
| Selection Sort | $O(n^2)$   | $O(1)$ | No      |
| Insertion Sort | $O(n^2)$   | $O(1)$ | Yes     |

A natural question arises: **Can we do better than  $O(n^2)$ ?**

Yes -- algorithms like Merge Sort and Quicksort run in  $O(n \log n)$ .

But today we ask a deeper question: **Is there a fundamental limit? Could someone someday invent an algorithm faster than  $O(n \log n)$ ?**

The answer is no. We will prove it today.

## Part 1: The Lower Bound

### Proving the Speed Limit of Comparison-Based Sorting

---

## What is a Comparison-Based Sort?

---

A **comparison-based sorting algorithm** determines the final sorted order using **only pairwise comparisons** between elements.

Every comparison asks one question: "Is  $a_i \leq a_j$ ?"

**All algorithms we have studied are comparison-based:**

- Bubble Sort, Selection Sort, Insertion Sort -- compare adjacent or distant pairs
- Merge Sort -- compares elements from two halves during the merge
- Quicksort -- compares elements against a pivot

**Non-comparison sorts exist** (Counting Sort, Radix Sort, Bucket Sort), but they require special assumptions about the data -- for example, that all keys are integers in a bounded range.

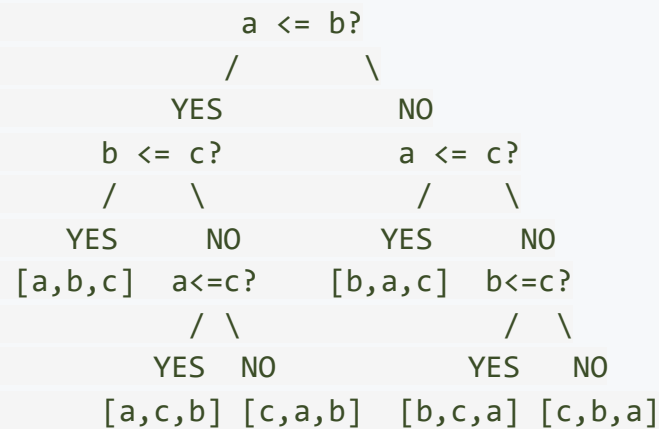
Today's question: **What is the fastest any comparison-based sort can possibly be?**

## The Decision Tree Model

Any comparison-based sort on  $n$  elements can be modeled as a **binary decision tree**.

- Each **internal node** represents a comparison: "Is  $a_i \leq a_j$ ?"
- Each **branch** represents the outcome: **Yes** (left) or **No** (right)
- Each **leaf** represents a unique permutation -- the final sorted arrangement

**Example: Sorting 3 elements [a, b, c]**



There are  $3! = 6$  leaves -- one for each possible ordering of a, b, c.

## The Key Insight: How Many Leaves Must There Be?

---

The algorithm must be able to **correctly sort every possible input**.

For  $n$  elements, there are  $n!$  possible orderings (permutations) of the input.

**Each leaf must correspond to exactly one permutation.** Therefore:

▮ The decision tree must have **at least  $n!$  leaves**.

Now we use a property of binary trees:

▮ A binary tree of **height  $h$**  has at most  $2^h$  leaves.

Putting these two facts together:

$$2^h \geq n!$$

Taking  $\log_2$  of both sides:

$$h \geq \log_2(n!)$$

## The Proof, Step 1: Why $n! \geq \left(\frac{n}{2}\right)^{n/2}$ ?

We have  $h \geq \log_2(n!)$ , so we need to find a simpler lower bound for  $\log_2(n!)$ .

**Strategy: Throw away the small factors of  $n$ !**

Write out  $n!$  and split it into two halves:

$$n! = \underbrace{n \times (n-1) \times \cdots \times \left(\frac{n}{2} + 1\right)}_{\text{large half: } n/2 \text{ factors, each } \geq n/2} \times \underbrace{\frac{n}{2} \times \cdots \times 1}_{\text{small half (discard)}}$$

Since we are finding a **lower bound**, we can drop the small half entirely -- it can only make the product smaller:

$$n! \geq \underbrace{\frac{n}{2} \times \frac{n}{2} \times \cdots \times \frac{n}{2}}_{n/2 \text{ times}} = \left(\frac{n}{2}\right)^{n/2}$$

**Concrete check with  $n = 8$ :**

$$8! = 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 40320$$

$$\left(\frac{8}{2}\right)^{8/2} = 4^4 = 256$$

Indeed  $40320 \geq 256$ . We kept only the 4 large factors (8, 7, 6, 5) and replaced each with the minimum value in that group (4). This gives a very loose but valid lower bound.

## The Proof, Step 2: Taking the $\log_2$ and Concluding

---

We have established:  $n! \geq \left(\frac{n}{2}\right)^{n/2}$

Since  $\log_2$  is a monotonically increasing function, taking  $\log_2$  of both sides preserves the inequality:

$$\log_2(n!) \geq \log_2 \left(\frac{n}{2}\right)^{n/2}$$

Expanding the right side:

$$\begin{aligned} \log_2 \left(\frac{n}{2}\right)^{n/2} &= \frac{n}{2} \cdot \log_2 \left(\frac{n}{2}\right) \quad [\text{power rule: } \log(x^k) = k \log x] \\ &= \frac{n}{2} \cdot (\log_2 n - \log_2 2) \quad [\text{quotient rule: } \log \frac{a}{b} = \log a - \log b] \\ &= \frac{n}{2} \cdot (\log_2 n - 1) \quad [\log_2 2 = 1] \\ &= \frac{n \log_2 n}{2} - \frac{n}{2} \end{aligned}$$

For large  $n$ , the  $-\frac{n}{2}$  term is negligible compared to  $\frac{n \log_2 n}{2}$ , so this is  $\Omega(n \log n)$ .

**Therefore we can conclude that:**  $h \geq \log_2(n!) \geq \Omega(n \log n)$ , therefore  $h = \Omega(n \log n)$ .

The height of the decision tree equals the worst-case number of comparisons. **Any comparison-based sort must make at least  $\Omega(n \log n)$  comparisons in the worst case.**

## What Does This Mean in Practice?

---

This is a **universal lower bound** -- not just a statement about known algorithms, but about all possible future algorithms too.

**No matter how clever your algorithm is**, if it can only compare elements pairwise, it cannot do better than  $\Omega(n \log n)$  comparisons in the worst case.

**Algorithms that achieve  $O(n \log n)$  worst case are asymptotically optimal:**

| Algorithm         | Worst Case     | Achieves Optimal? |
|-------------------|----------------|-------------------|
| Bubble Sort       | $O(n^2)$       | No                |
| Selection Sort    | $O(n^2)$       | No                |
| Insertion Sort    | $O(n^2)$       | No                |
| <b>Merge Sort</b> | $O(n \log n)$  | <b>Yes</b>        |
| <b>Heap Sort</b>  | $O(n \log n)$  | <b>Yes</b>        |
| Quicksort         | $O(n^2)$ worst | Not in worst case |

From CLRS: "*The  $\Omega(n \log n)$  bound tells us that merge sort and heap sort are asymptotically optimal comparison sorts.*"

## How Non-Comparison Sorts Beat the Bound

---

Counting Sort, Radix Sort, and Bucket Sort can achieve  $O(n)$  or  $O(n \log n)$  with better constants -- **but they cheat, in a good way:**

- They do not rely solely on comparisons
- They exploit the **structure of the keys** (e.g., integer values, bounded range)
- They are not modeled by the decision tree -- so the lower bound does not apply

**The trade-off:** They only work for specific data types. Merge Sort works for any data where a comparison ( $\leq$ ) is defined -- integers, floats, strings, even custom objects.

Today's algorithm -- **Merge Sort** -- achieves the optimal  $O(n \log n)$  for all comparison-based inputs, with no special assumptions about the data.

## Part 2: Merge Sort

### Divide, Conquer, and Merge

---

## The Divide and Conquer Paradigm

---

Merge Sort is our first algorithm built on a powerful strategy called **Divide and Conquer**. It has three steps:

### 1. Divide

Break the problem into smaller subproblems of the same type.

### 2. Conquer

Solve each subproblem recursively. The base case is a subproblem small enough to solve directly.

### 3. Combine

Merge the solutions of the subproblems into a solution for the original problem.

We have already seen this idea with Binary Search: divide the search space in half, conquer one half recursively, and there is nothing to combine.

Merge Sort is more powerful -- the **combine step** (merging) is where the real work happens.

## The Merge Sort Idea

---

The key insight behind Merge Sort:

**Merging two sorted arrays into one sorted array takes only  $O(n)$  time.**

So the strategy is:

1. **Divide** the array into two halves
2. **Recursively sort** each half (trust the recursion!)
3. **Merge** the two sorted halves -- this takes  $O(n)$

**Analogy:** Imagine you have two decks of cards, each already sorted face-up. To combine them into one sorted deck, you repeatedly compare the top card of each deck and take the smaller one. You never need to look further than the top of each pile -- one forward pass through both decks.

**Invented by John von Neumann in 1945** -- one of the oldest algorithms still in widespread production use today.

## **Part 3: The Merge Procedure**

### The Heart of Merge Sort

---

## How Merging Works

---

Given two sorted subarrays, we want to combine them into a single sorted array.

**Setup:** Two sorted halves -- Left and Right. One output array.

**Algorithm:**

1. Use two pointers, **i** (into Left) and **j** (into Right)
2. Compare **L[i]** and **R[j]** -- copy the smaller one to the output, advance that pointer
3. When one array is exhausted, copy all remaining elements of the other

```
L = [2, 5, 8]      R = [1, 4, 7]
    ^              ^
    i              j
```

The crucial observation: we never need to go backward. One left-to-right pass through both arrays is sufficient. This is what makes merging  $O(n)$  -- we touch each element exactly once.

## Merge: Pseudocode

```
Merge(A, left, mid, right):  
    Create temp arrays:  
        L = A[left .. mid]  
        R = A[mid+1 .. right]  
  
    i = 0, j = 0, k = left  
  
    while i < |L| and j < |R|:  
        if L[i] <= R[j]:      // <= ensures stability!  
            A[k] = L[i]  
            i = i + 1  
        else:  
            A[k] = R[j]  
            j = j + 1  
        k = k + 1  
  
    // Copy any remaining elements  
    while i < |L|: A[k++] = L[i++]  
    while j < |R|: A[k++] = R[j++]
```

**Key detail:** Using `<=` (not `<`) when comparing `L[i]` and `R[j]` ensures **stability** -- equal elements from the left half are always placed before equal elements from the right half.

## Merge: Complete Traced Example

---

Merge  $L = [2, 5, 8]$  and  $R = [1, 4, 7]$

```
i=0, j=0:  Compare  2 vs 1   -->  1 smaller.  Copy 1.   Output: [1]
i=0, j=1:  Compare  2 vs 4   -->  2 smaller.  Copy 2.   Output: [1, 2]
i=1, j=1:  Compare  5 vs 4   -->  4 smaller.  Copy 4.   Output: [1, 2, 4]
i=1, j=2:  Compare  5 vs 7   -->  5 smaller.  Copy 5.   Output: [1, 2, 4, 5]
i=2, j=2:  Compare  8 vs 7   -->  7 smaller.  Copy 7.   Output: [1, 2, 4, 5, 7]
i=2, j=3:  j exhausted.      Copy remaining L: [8].    Output: [1, 2, 4, 5, 7, 8]
```

**Total comparisons: 5** (which is  $n - 1$  in the worst case, where  $n = 6$ )

Notice:

- We never went backward -- purely left to right through both arrays
- Every element was copied exactly once
- The result is perfectly sorted

## Why Is Merging $O(n)$ ?

---

At each step we copy one element to the output. There are  $n$  elements total.

Each element is copied **exactly once** -- never revisited.

Therefore, the merge procedure is always  $\Theta(n)$  -- it costs the same regardless of the arrangement of elements, as long as both halves are sorted.

**This linearity is the key.** In contrast:

- Sorting from scratch takes  $O(n^2)$  or  $O(n \log n)$
- But merging two already-sorted arrays is just  $O(n)$

Merge Sort exploits this by breaking the sorting problem into smaller and smaller pieces until each piece is trivially sorted (one element), then merging them back up the tree.

## **Part 4: The Complete Merge Sort**

### Recursion, Traces, and Analysis

---

## The Recursive Algorithm

```
MergeSort(A, left, right):  
    if left < right:                // Base case: one element, stop  
        mid = (left + right) / 2  
        MergeSort(A, left, mid)    // Sort left half  
        MergeSort(A, mid + 1, right) // Sort right half  
        Merge(A, left, mid, right) // Combine
```

### Reading this algorithm:

- **Base case:** `left == right` -- a single-element subarray is already sorted. Return immediately.
- **Recursive case:** Find the midpoint, sort each half independently, then merge.

**Trust the recursion:** When `MergeSort(A, left, mid)` returns, the left half is guaranteed to be sorted. Same for the right half. Then `Merge` combines them in  $O(n)$ .

The pivot of the algorithm's correctness is that the merge of two sorted halves is always correct and  $O(n)$ .

## Complete Traced Example: Splitting Phase

---

Array: [38, 27, 43, 3, 9, 82, 10]

Level 0 (original):

[38, 27, 43, 3, 9, 82, 10]

split at mid=3

Level 1 (two halves):

[38, 27, 43, 3]            [9, 82, 10]

Level 2 (four quarters):

[38, 27]   [43, 3]        [9, 82]   [10]

Level 3 (base cases):

[38] [27] [43] [3]        [9] [82] [10]

We split until every subarray has exactly one element. Single elements are trivially sorted.

Now we **unwind the recursion** -- merging back up the tree.

## Complete Traced Example: Merging Phase

---

Level 3 --> Level 2: merge pairs

Merge [38],[27] --> [27, 38]

Merge [43],[3] --> [3, 43]

Merge [9],[82] --> [9, 82]

[10] stays as-is (only one element on right)

Level 2 --> Level 1: merge halves

Merge [27,38] and [3,43] --> [3, 27, 38, 43]

Merge [9,82] and [10] --> [9, 10, 82]

Level 1 --> Level 0: final merge

Merge [3,27,38,43] and [9,10,82]

--> [3, 9, 10, 27, 38, 43, 82]

**Done!** The array is sorted.

## Visualizing the Recursion Tree

Each level of the recursion tree does  $O(n)$  work total:

|          |                             |                                                                              |
|----------|-----------------------------|------------------------------------------------------------------------------|
| Level 0: | [38,27,43,3,9,82,10]        | <-- 1 merge of 7 elements<br>Work: $O(7) = O(n)$                             |
| Level 1: | [38,27,43,3] [9,82,10]      | <-- 2 merges, $\sim n/2$ elements each<br>Work: $O(4) + O(3) = O(n)$         |
| Level 2: | [38,27][43,3] [9,82][10]    | <-- 4 merges, $\sim n/4$ elements each<br>Work: $O(2)+O(2)+O(2)+O(1) = O(n)$ |
| Level 3: | [38][27][43][3] [9][82][10] | <-- base cases, $O(1)$ each<br>Work: $O(n)$                                  |

**Number of levels:**  $\log_2 n$  (we halve the size at each level)

**Work per level:**  $O(n)$

**Total:**  $O(n) \times O(\log n) = O(n \log n)$

## Merge Sort Complexity Analysis

---

### Recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

- $2T(n/2)$ : two recursive calls, each on half the input
- $O(n)$ : the merge step

### Solving by Master Theorem (Case 2):

$$T(n) = O(n \log n)$$

### Summary of properties:

| Property     | Value                                                           |
|--------------|-----------------------------------------------------------------|
| Best case    | $O(n \log n)$                                                   |
| Average case | $O(n \log n)$                                                   |
| Worst case   | $O(n \log n)$                                                   |
| Space        | $O(n)$ -- the temporary arrays during merging                   |
| Stable?      | Yes -- the <code>&lt;=</code> in the merge step preserves order |
| In-place?    | No -- requires $O(n)$ auxiliary space                           |

## The Guarantee: No Surprises

---

One of Merge Sort's greatest strengths is **predictability**.

- Bubble Sort: best  $O(n)$ , but  $O(n^2)$  if you're unlucky
- Insertion Sort: best  $O(n)$ , but  $O(n^2)$  on reverse-sorted input
- Quicksort (next lecture): average  $O(n \log n)$ , but  $O(n^2)$  in the worst case

**Merge Sort: always  $O(n \log n)$ .** No exceptions. No bad inputs. No worst-case surprises.

This predictability makes Merge Sort the algorithm of choice when:

- You cannot afford unpredictable worst-case behavior
- You are sorting linked lists (merge sort on linked lists requires no extra space)
- You need a **stable**  $O(n \log n)$  sort (Heap Sort is not stable)

**The only cost:**  $O(n)$  extra memory for the temporary arrays.

## Merge Sort is Stable: A Verification

Recall that stability means equal elements retain their original relative order.

**Why Merge Sort is stable:** In the merge step, when `L[i] == R[j]`, we use `<=`:

```
if L[i] <= R[j]:  
    copy L[i]    // Left element comes first!
```

Equal elements from the **left half** are always placed before equal elements from the **right half**. Since the left half corresponds to elements that appeared earlier in the original array, their relative order is preserved.

**Example:** Merging [3, 5a, 7] and [1, 5b, 9]

```
1 < 3 -> copy 1.   Output: [1]  
3 < 5b -> copy 3.  Output: [1, 3]  
5a <= 5b -> copy 5a (LEFT first!). Output: [1, 3, 5a]  
5b < 7 -> copy 5b. Output: [1, 3, 5a, 5b]  
...  
Final: [1, 3, 5a, 5b, 7, 9] -- 5a still before 5b. Stable!
```

## Part 5: Merge Sort vs Simple Sorts

How Far We Have Come

---

## The Updated Complexity Table

| Algorithm         | Best                            | Average                         | Worst                           | Space                    | Stable?    | In-Place? |
|-------------------|---------------------------------|---------------------------------|---------------------------------|--------------------------|------------|-----------|
| Bubble Sort       | $O(n)$                          | $O(n^2)$                        | $O(n^2)$                        | $O(1)$                   | Yes        | Yes       |
| Selection Sort    | $O(n^2)$                        | $O(n^2)$                        | $O(n^2)$                        | $O(1)$                   | No         | Yes       |
| Insertion Sort    | $O(n)$                          | $O(n^2)$                        | $O(n^2)$                        | $O(1)$                   | Yes        | Yes       |
| <b>Merge Sort</b> | <b><math>O(n \log n)</math></b> | <b><math>O(n \log n)</math></b> | <b><math>O(n \log n)</math></b> | <b><math>O(n)</math></b> | <b>Yes</b> | <b>No</b> |

Merge Sort is the first algorithm in our course that breaks the  $O(n^2)$  barrier -- and it does so in all cases, not just on lucky inputs.

The cost: it requires  $O(n)$  extra memory. For the next lecture's algorithm (Quicksort), we will see if we can sort in-place while still achieving  $O(n \log n)$  on average.

## Scale Matters: $n^2$ vs $n \log n$

---

For large  $n$ , the difference between  $O(n^2)$  and  $O(n \log n)$  is dramatic.

Assume each comparison takes 1 nanosecond:

| $n$       | $n^2$ operations    | $n \log_2 n$ operations | Time saved     |
|-----------|---------------------|-------------------------|----------------|
| 1,000     | 1,000,000           | ~10,000                 | 100x faster    |
| 100,000   | 10,000,000,000      | ~1,700,000              | ~6,000x faster |
| 1,000,000 | $10^{12}$ (~16 min) | ~20,000,000 (~20ms)     | 50,000x faster |

At  $n = 1,000,000$  elements:

- Insertion Sort (worst case): about **16 minutes**
- Merge Sort (any case): about **20 milliseconds**

The lower bound we proved guarantees that **no comparison sort can beat Merge Sort's asymptotic complexity.**

## Visual Tool: Watch Merge Sort in Action

---

VisuAlgo -- Sorting Algorithm Visualizer:

<https://visualgo.net/en/sorting>

Select "**Merge Sort**" from the algorithm menu. Watch how:

- The array is repeatedly divided in half (splitting phase)
- Single elements at the bottom are trivially sorted
- Merging builds the sorted result from the bottom up
- Each merge step takes elements in sorted order from two piles

Try this comparison: sort the same array with **Insertion Sort** and then with **Merge Sort**, and observe how many operations each one performs. The difference is striking for arrays of even moderate size.

## **Part 6: Summary and What is Next**

## Today's Key Concepts

---

### The Lower Bound:

- Any comparison-based sort requires at least  $\Omega(n \log n)$  comparisons in the worst case
- Proven using the decision tree model: the tree needs  $n!$  leaves, a binary tree of height  $h$  has at most  $2^h$  leaves, so  $h \geq \log_2(n!) = \Omega(n \log n)$
- This is a universal, absolute limit -- no future algorithm can break it

### The Merge Procedure:

- Merging two sorted arrays of total size  $n$  takes exactly  $O(n)$  time
- Uses two pointers, one pass through both arrays
- Using `<=` (not `<`) preserves stability

### Merge Sort:

- Divide: split at the midpoint. Conquer: sort each half recursively. Combine: merge in  $O(n)$
- Recurrence:  $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$
- Guaranteed  $O(n \log n)$  in all cases -- the first asymptotically optimal sort we have studied

## Review Questions

---

1. We proved that any comparison-based sort needs  $\Omega(n \log n)$  comparisons. Counting Sort can sort  $n$  integers in  $O(n)$  time. Is this a contradiction? Explain.
2. Trace Merge Sort on the array [5, 2, 8, 3, 9, 1]. Show the array state at every split and every merge step.
3. Merge  $L = [1, 4, 6, 9]$  and  $R = [2, 3, 7, 8]$  step by step. How many comparisons are made in total?
4. Why does the merge procedure use `<=` instead of `<` when comparing  $L[i]$  and  $R[j]$ ? What would change if we used strict `<`?
5. Merge Sort requires  $O(n)$  extra space. Can you think of a scenario where this is a serious problem? How might you work around it?
6. A database needs to sort 500 million customer records by last name. It must guarantee a response time of under 10 seconds. Which sort would you recommend -- Insertion Sort, Merge Sort, or Quicksort? Justify your answer.

## What Comes Next

---

**Lecture 24** introduces **Quicksort** -- arguably the most important sorting algorithm in practice.

Quicksort takes a different divide-and-conquer approach: instead of splitting the array at the midpoint, it **partitions** elements around a chosen **pivot**, placing all smaller elements to the left and all larger elements to the right. Then it recursively sorts each side.

Quicksort achieves  $O(n \log n)$  **on average** and sorts **in-place** -- no auxiliary array needed. In practice, it is often faster than Merge Sort because of better cache behavior and lower overhead, despite having an  $O(n^2)$  worst case.

We will study two partitioning schemes:

- **Lomuto Partition:** elegant and easy to implement correctly
- **Hoare Partition:** Tony Hoare's original design, fewer swaps on average

**The era of  $O(n^2)$  sorts is behind us. Now we explore the algorithms that power real-world software.**